

Step 3: Run Prompts 5 Times

Sentiment & extraction: The output is identical across all five runs. Both the format and content stay the same because these tasks have clear, limited answers, so changing the temperature does not noticeably affect the results.

Product description: The overall structure is consistent in every run, but some details change. The product name is different between runs because it was not specified in the prompt. Some features appear in certain runs but not others, and details like battery life also vary. Since this information was not provided, the model generated its own details.

Overall, classification and extraction tasks were very consistent, while the product description varied more. A better prompt should specify the product name, required features, and important specifications.

Step 4: Run Prompts 10 Times

The product description followed two main patterns. Five runs generated 'SwiftClick Wireless Mouse' and five generated 'Wireless Precision Mouse.' The model switches between these two common responses instead of producing completely random outputs.

Some details still changed. Features like adjustable DPI and customizable buttons did not appear every time, and the battery life was described differently because these details were not included in the prompt.

Unlike sentiment analysis and data extraction, all 10 product descriptions were different. This shows that open-ended tasks naturally produce more variation.

Step 5: Run Prompts 15 Times and Create Failure Analysis

After 15 runs, the same pattern continued. Five responses used 'SwiftClick Wireless Mouse' and ten used 'Wireless Precision Mouse.' The ratio changed, but the model still relied on the same two templates.

Most responses were unique (13 out of 15). Battery life was described in different ways, and a few extra details, like a 33-foot wireless range, appeared only once.

Overall, the structure stayed similar, but the names, features, and specifications changed from run to run. The more freedom the prompt gives the model, the more variation appears.

Failure Analysis Table

Prompt	Consistency (n=5 / n=10 / n=15)	Unique Responses (n=15)	Primary Failure Pattern(s)	Severity
Sentiment Analysis	100% / 90% / 67%	3 / 15	Wording drift ("This" vs "The customer message"); 1 run dropped bold/caps formatting on labels	Low
Data Extraction	100% / 100% / 100%	1 / 15	None observed, output identical across all 30 runs tested (n=5, 10, 15)	None
Product Description	60% / 50% / 67%	13 / 15	Bimodal product naming (SwiftClick vs. Wireless Precision Mouse); invented battery-life duration differs almost every run; one-off fabricated specs (33 ft range, left/right-handed claim) that never repeat; optional features (DPI settings, programmable buttons, OS compatibility) appear in only 1-2 runs	High

Specific Failure Patterns by Prompt

Sentiment Analysis: Fully consistent at 5 runs (100%), but drops to 67% dominant-response consistency by 15 runs. The classification itself never changes (always positive/satisfaction), but the exact wording drifts between "This customer message..." and "The customer message...", and one run lowercased the bold labels instead of capitalizing them. This is surface-level phrasing drift, not a reasoning failure. If downstream code parses this output as a free-text label rather than exact string match, it is not a functional failure. If exact-string matching is required (a fixed enum, for example), this would break 5 of 15 runs.

Data Extraction: Zero failures across all 30 runs tested (5, 10, and 15). Every field (order number, date, delivery speed, packaging condition) was extracted identically every time, with identical formatting. This is the only prompt of the three with no observed inconsistency at any sample size, confirming that narrowly-scoped extraction tasks with an explicit, bounded set of fields to pull are the most reliable prompt type under repeated sampling.

Product Description: The least reliable of the three prompts, and the only one showing multiple distinct failure types simultaneously. First, identity instability: the model invents a different product name each run because none was given, converging on two competing templates ("SwiftClick Wireless Mouse" vs. "Wireless Precision Mouse") rather than one stable answer; the split ratio itself is unstable (60/40 at n=5, 50/50 at n=10, 33/67 at n=15). Second, fact fabrication: battery life is stated as "months," "12 months," "weeks," or omitted entirely, with no single run's claim consistently reproduced. Third, one-off hallucinations: a "33 ft wireless range" and "left/right-handed" claim each appeared in exactly 1 of 15 runs and never recurred, invented detail with no grounding in the prompt. Fourth, unstable feature inclusion: bonus features like DPI settings or programmable buttons appear in only 1-2 of 15 runs. 13 of 15 full responses were entirely unique. Root cause: the prompt leaves product name, features, and specs unconstrained, so the model fills every gap independently each run.

Cross-Prompt Takeaway: Consistency scales with how constrained the task is. Classification and extraction tasks, which have one defensible correct answer, stay at or near 100% consistency even as sample size grows. Open-ended generation, which leaves names, features, and specs unspecified, degrades further as sample size increases; more runs simply reveal more of the model's latent variability, including rare one-off hallucinations that a 5-run test would never catch. The fix for the product prompt is to remove the open-endedness: supply the product name, specify which features must appear, and either provide the battery spec or explicitly instruct the model not to state a battery duration if none is given.

Part 3, Iteration 2: Few-Shot and Chain-of-Thought, Version 3 vs. Version 1

Step 9 added 2-3 few-shot examples (Negative/Neutral/Positive) to the sentiment prompt. Step 10 added chain-of-thought reasoning to the extraction prompt, with the final answer isolated after a "Final JSON:" marker. Step 11 combined few-shot examples with the structural, length, and style constraints from Iteration 1 for the product description prompt. All three were re-tested at n=15, same as the v1 baseline.

Prompt	v1 Consistency (n=15)	v3 Consistency (n=15)	Technique Used	Result
Sentiment Analysis	67% (3 unique responses)	100% (1 unique: "Positive")	Few-shot (3 examples)	Fully solved, exceeds the 80%+ target
Data Extraction	100% (already at ceiling)	100% full-text; 100% JSON-parseable	Chain-of-thought	No regression; adds a reasoning trace at no consistency cost
Product Description	33/67 naming split; 13/15 unique	Headline 100% consistent; 4/4 bullets every run; 12/15 unique	Few-shot + structure	Structure/identity solved; fine-grained facts still vary

Findings by Prompt

Sentiment: Few-shot fully closed the gap seen in the failure analysis. All 15 runs returned the identical single word "Positive," eliminating both the "This" vs. "The customer message" wording drift and the one-off casing failure observed in v1. This result beats the lab's 80%+ consistency target outright.

Data Extraction: Chain-of-thought had nothing to fix here since v1 was already at 100% consistency, but it is a clean result nonetheless: the reasoning text was identical across all 15

runs, and the JSON extracted from after the "Final JSON:" marker parsed successfully every time (15/15). The added structure did not cost any reliability, though it does require a regex extraction step that a plain-JSON prompt (v2) did not need.

Product Description: The most nuanced result. Macro-level instability from v1, the competing "SwiftClick" vs. "Wireless Precision Mouse" identities and inconsistent header format, is completely gone: all 15 runs now share one headline ("Effortless Navigation at Your Fingertips") and exactly 4 bullet points. Word count also improved, landing in a 105-120 range (avg 111) versus v2's flat 87 words every run, though only 2 of 15 runs fell inside the strict 120-150 target. At the micro level, however, invented facts persist: battery life fragments into 7 different phrasings across 15 runs, and the fabricated "33 feet" wireless-range claim actually appears more often than in v1 (5/15 vs. 1/15), likely because the worked example primed the model to state a specific number rather than staying vague. Few-shot and structure fix which template the model uses; they do not stop it from inventing specific unconstrained numbers within that template.

Cross-Iteration Takeaway: Prompt engineering technique and failure type need to be matched. Few-shot examples are highly effective at collapsing format and identity ambiguity (sentiment's wording drift, product's naming split) because they show the model the exact shape of a correct answer. They are less effective at stopping fact fabrication, because an example naturally includes a specific detail (a battery duration, a wireless range) that the model can imitate rather than recognizing as a placeholder. To fully solve the product prompt's remaining failure, the next iteration should either supply the real spec values directly or explicitly instruct the model to omit any detail not provided in the prompt.

Part 5, Step 13: Final Evaluation and Comparison (v1 to v4)

All three prompts were run 15 times each per version (10 for v2, tested at a smaller sample), across the full iteration history: v1 (baseline, unconstrained), v2 (structure/format constraints), v3 (few-shot and/or chain-of-thought), and v4 (final, task-specific refinements). The table below reports the consistency percentage and unique-response count for each version of each task.

Task	Version	N	Technique	Consistency %	Unique Responses
Sentiment Analysis	v1 (baseline)	15	None	67%	3
Sentiment Analysis	v2 (single-word constraint)	10	Format constraint	100%	1
Sentiment Analysis	v3 (few-shot)	15	Few-shot (3 examples)	100%	1
Sentiment Analysis	v4 (final)	15	Few-shot + percentage breakdown format	100%	1
Data Extraction	v1 (baseline)	15	None	100%	1

Data Extraction	v2 (structured JSON)	10	Explicit JSON format	100%	1
Data Extraction	v3 (chain-of-thought)	15	CoT reasoning	100%	1
Data Extraction	v4 (final)	15	CoT + expanded category options	100%	2
Product Description	v1 (baseline)	15	None	67%	13
Product Description	v2 (structure+length+style)	10	Format constraints	100%	2
Product Description	v3 (few-shot+structure)	15	Few-shot + constraints	100%	12
Product Description	v4 (final, premium tone)	15	Few-shot + constraints + tone spec	80%	10

Improvement Summary: v1 (Baseline) vs. v4 (Final)

Task	v1 Consistency	v4 Consistency	Change	v1 Unique	v4 Unique	Unique Reduction
Sentiment Analysis	67%	100%	+33 pp	3	1	2
Data Extraction	100%	100%	0 pp	1	2	-1
Product Description	67%	80%	+13 pp	13	10	3

Task-Specific Secondary Metrics

Product Description, Word-Count Compliance: Word-count compliance (target: 120-150 words) climbed steadily as constraints tightened, even though it was never fully solved. The table below tracks the percentage of runs landing inside the target range at each version.

Version	N	Word-Count Constraint	In-Range %
v1	15	None	N/A
v2	10	120-150 words	0%
v3	15	120-150 words	13%
v4	15	120-150 words	53%

Data Extraction, JSON Parseability: Raw JSON parseability moved in the opposite direction of what might be expected. The table below tracks whether the raw model output parses directly with a JSON parser, and if not, whether it parses after isolating or cleaning the JSON block.

Version	N	Output Format	Raw Parse %	Parse After Extraction %
v1	15	Free text	N/A	N/A
v2	10	Plain JSON	100%	N/A
v3	15	CoT + isolated JSON block	0%	100%
v4	15	CoT + fenced json block	0%	100%

v2's plain-JSON output parsed successfully 100% of the time with no processing. Once chain-of-thought reasoning was introduced in v3, raw parsing dropped to 0% (reasoning text precedes the JSON), recovering to 100% only after isolating the JSON block. v4 kept that same 0%/100% pattern but added an unrequested fenced json code block not seen in v3, meaning the extraction step needs to strip the fence as well as isolate the block.

Key Findings

Sentiment: Improved the most cleanly: 67% to 100% consistency (a 33 percentage point gain), with unique responses dropping from 3 to 1. The single-word constraint (v2) alone was enough to reach the ceiling; few-shot (v3) and the percentage-breakdown format (v4) preserved that consistency while changing the output shape.

Extraction: Was already at 100% consistency in v1, so consistency was never the bottleneck. What changed across versions was parseability: plain JSON (v2) parsed raw every time, while the chain-of-thought versions (v3, v4) require an extraction step, and v4 additionally introduced code fences that v3 did not have.

Product Description: Shows the clearest trade-off in the whole project. Word-count compliance improved substantially (0% to 13% to 53%) as constraints tightened, but headline and naming consistency actually dropped from 100% (v2, v3) to 80% (v4) once a tone instruction was added. Tightening one dimension loosened another; this should be reported as a genuine trade-off, not a pure win.